

NYCU-ECE DCS-2026

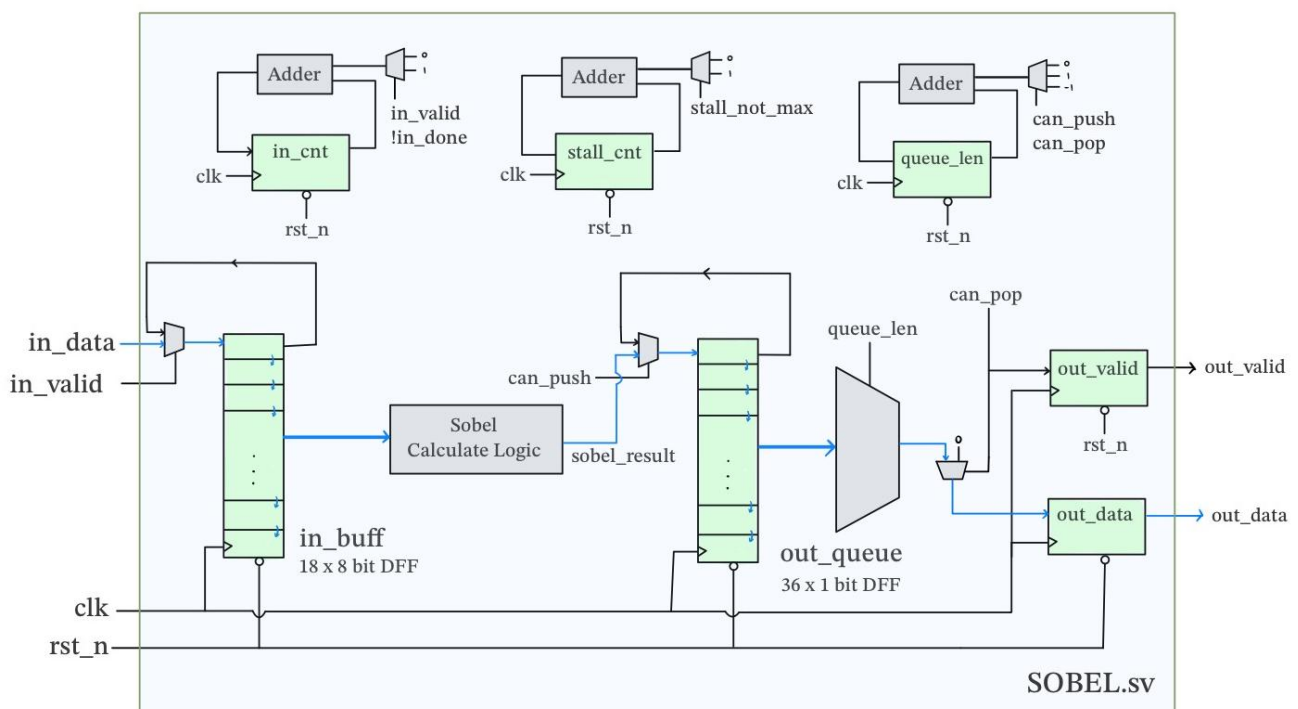
HW02 Report

Name: 楊庭瑞

Student ID:113511266

Server Account: dcs151

Architecture



Design Methodology

● 設計架構:

將 Sequential 部分和 Combinational 部分分開，將每個 DFF 前後變數都設成 `*_cs`, `*_ns`，在 Sequential 部分只要使用 `*_cs <= *_ns` 就好，而 Combinational 則利用現有的 `*_cs` 經過邏輯後算出新的 `*_ns`。

1. 將 `indata` 傳入 `in buff`，`in_buff` 設成 18 個，一個一個吃進來，在 Shift 往下移，可以計算的時候就根據九宮格計算出 `sobel_result`
2. 算完的 `sobel_result` 存入隊列中 `out_queue`，每次存入後再 Shift 移位，輸出根據 `queue_len` 決定要將哪個放成 `out_data`。
3. 控制隊列輸入 `can_push` 的判斷為可計算出 Sobel 的最小 `in_cnt`，`can_pop` 則需要確保在沒有 `in_valid` 且隊列有元素存在，及輸入結束或還沒 Stall 完才可以輸出。

```
assign can_push = (in_cnt_cs[5:3] >= 3'd2 && in_cnt_cs[2:0] >= 3'd2 && in_valid);  
assign can_pop = ((!in_valid) && has_queue) && (input_done || stall_not_max);
```

- 共使用三個 Counter 獨立寫一個 always_comb，避免所有判斷都寫在同一個 always block，其中 queue_len 根據 push or pop 進行 +1 或 -1。

- 面積優化

- 一開始我直接開一個 8*8 的大 in_buff，想說慢慢把 input 吃進來，再算完就好，但這樣面積會超大，就改成使用最少的 in_buff，18 個其實就夠算完全部了，利用 Shift Register 逐步移位去計算節省了很多面積。
- 原本利用兩個 Counter，一個紀錄算完了幾個 Sobel，一個紀錄 Output_cnt 了幾個，但這樣子寫入跟寫出都要使用到 36 to 1 的 mux 去選擇，後來改成 queue 的想法，push 進第一格再 Shift，讀出的時候再利用 mux 去選擇，這樣子面積也節省了許多。

```
// Counter and queue Length Control Combinational Logic
always_comb begin
    // input cnt
    in_cnt_ns = in_cnt_cs;
    if (in_valid) begin
        if (!input_done) in_cnt_ns = in_cnt_cs + 7'd1;
        else in_cnt_ns = 1;
    end

    //stall_cnt
    stall_cnt_ns = stall_cnt_cs;
    if (in_valid) stall_cnt_ns = 3'd0;
    else if (stall_not_max) stall_cnt_ns = stall_cnt_cs + 3'd1;

    //queue_len
    queue_len_ns = queue_len_cs;
    if (can_push) queue_len_ns = queue_len_cs + 6'd1;
    if (can_pop) queue_len_ns = queue_len_cs - 6'd1;
end
```

Reflection

這次的作業不同於 HW01 的純 Combinational 邏輯，由於太久沒碰邏設中學到的 Sequential 的概念，導致我在寫作業的時候有點挫折，我一開始在寫 Sequential 的 Block 裡面都會加一些邏輯，導致非常難想，後來使用 *_cs, *_ns 的 Coding Style 變得比較順利，程式碼也變得比較乾淨。

這次作業也非常有趣，一開始我用很笨的方法，直接 8*8 爆開，也不趁 input stall 期間偷偷輸出，latency 就超級大，後來慢慢一直優化，到最後不知道怎麼優化，每次想到新的架構，演算法之類的都讓我有滿滿的成就感，但在壓面積部分我還是不太理解，為什麼有時候腦袋覺得變小，結果跑出來其實是變大，讓我非常疑惑，也意識到自己還有很多需要學習。

AI-use Log

If AI was used, please include the following details:

- What tools/models you used? (e.g., Gemini, ChatGPT, Copilot, ...)
GPT-5.3-Codex, Gemini 3.1 pro
- Key Prompts: 3–5 primary prompts (paste the original text or conversation link)
(貼程式碼) 幫我檢查是否有語法錯誤
(貼程式碼) 這一段是否可以優化面積或結構，或是改成...去實作
(貼程式碼) 出現了.....的問題應該怎麼辦才好(貼 nwave 波型及敘述何處有誤)。
- What and why you adopt or reject?
確保 AI 生出來的 code 自己看得懂，也認為比現在版本的還要好，盡可能避免在看不懂的時候直接採用。